1. Let $SA[1], SA[2], \ldots, SA[n]$ be the suffix array of a word $w[1..n]$ and let p be any string. Provide formal proofs of the following two statements:
   1. All suffixes $w[i..n]$ such that p is a prefix of $w[i..n]$ constitute a contiguous range in the suffix array.
   2. Let $SA[i], SA[i+1], \ldots, SA[j]$ be the aforementioned contiguous range. Then, $SA[i-1] < p \leq SA[i]$ (where $<$ denotes the lexicographical comparison).
2. Given a permutation on $\{1, 2, \ldots, n\}$, we want to find a word $w \in \Sigma^n$ such that its suffix array SA_w is exactly the given permutation.
   (a) Show a linear time algorithm solving the problem for $\Sigma = \{a, b\}$.
   (hard) (b) Show a linear time algorithm solving the problem for $\Sigma = \{a, b, \ldots, z\}$.
3. Show how to compute the values $lcp[2], lcp[3], \ldots, lcp[n]$ in $\mathcal{O}(n)$ time. You might want to use the following property: $lcp[SA^{-1}[i]] - 1 \leq lcp[SA^{-1}[i+1]]$.
4. Show how to use the suffix array (together with the lcp array) to find, given $s[1..n]$ and $t[1..n]$, the longest string that occurs in s but not in t in $\mathcal{O}(n)$ time.
5. A set $B \subseteq [0, t)$ is called a difference cover modulo t when, for every $x, y \in [0, t)$, there exists $\delta \in [0, t)$ such that $x + \delta \pmod t$ and $y + \delta \pmod{}$ belong to B. Construct a difference cover of size $\mathcal{O}(\sqrt{t})$. What is the smallest c such that you can obtain a difference cover of size $c\sqrt{t} + \mathcal{O}(1)$?
6. Consider a trie T of size n (this is just a tree, where each edge is labeled with a single character, and the edges outgoing from a node have pairwise distinct labels). For a node $v \in T$, we define $s(v)$ to be the string obtained by reading the labels of the edges from v to the root. We want to lexicographically sort the strings $s(v)$, for all $v \in T$. For T being a path, this is exactly the suffix array. Show how to generalise the algorithm from the lecture to work for any T in $\mathcal{O}(n)$ time.
7. (hard) Suffix array built for a text of length n allows locating the occurrences of a pattern of length m in $\mathcal{O}(m + \log n)$. Design a linear-space structure that supports such queries in $\mathcal{O}(m + \log |\Sigma|)$ time, where Σ is the alphabet.